



= SPEC-1: Overlay Alignment Aid App :sectnums: :toc:

== Background

This app addresses the need for spatial alignment between a previously captured photo of a location and the current camera view of that same location. It is particularly useful in scenarios such as construction progress tracking, restoration work, or creative photography where consistent framing is necessary over time. By overlaying the prior image directly onto the live camera feed with adjustable opacity, the user can visually align both views for accurate and repeatable captures.

== Requirements

- Must Have:
 - User can select a previously captured image from the device's gallery.
 - Live camera preview with a semi-transparent overlay of the selected image.
 - Ability to adjust overlay opacity using a slider.
 - Support for capturing a new image aligned with the overlay.
 - Save captured (after) image to local storage with metadata (e.g., timestamp).
- Should Have:
 - UI controls to reposition and scale the overlay manually for better alignment.
 - Grid or alignment guides over the live camera feed.
 - Option to group "before" and "after" images in the app.
- Could Have:
 - Side-by-side display of before/after images after capture.
 - Optional GPS metadata tagging for context.
 - Simple image enhancement (brightness, contrast) for overlay.
- Won't Have:
 - Cloud storage, user login, or sync features in the MVP.
 - Automatic alignment or AR features in the MVP.

== Method

The application is composed of the following major components:

- ImageSelector: Allows users to select a prior image from the gallery.
- OverlayPreviewActivity: Displays a full-screen alignment aid with a transparent overlay of the selected image on top of the live camera view (visible behind the app).
- OverlayController: Allows the user to reposition, scale, and adjust the opacity of the overlay image using on-screen gestures and sliders.

- InstructionsOverlay: Optional guidance text and alignment grid to help user match framing.
- The app does not invoke or interact with the camera or store the after image.

=== User Flow

[plantuml]

```
@startuml
actor User
User --> ImageSelector : Select "before" image from gallery
User --> OverlayPreviewActivity : Opens overlay preview screen
User --> OverlayController : Adjust opacity / position / scale
User --> SystemCameraApp : Manually open and take photo
@enduml
```

=== Android Architecture

- Kotlin with Jetpack Compose or XML.
- Uses `Intent.ACTION_PICK` to select image from the gallery.
- Displays full-screen overlay using:
- A transparent background activity
- `FLAG_LAYOUT_NO_LIMITS` + `FLAG_NOT_TOUCH_MODAL` + `TYPE_APPLICATION_OVERLAY` for overlaying while camera is visible behind
- Overlay controls:
- Slider for opacity (0–100%)
- Gesture listeners for drag and pinch-zoom
- No image capture or file storage in the app

=== Technical Design

- OverlayPreviewActivity uses `WindowManager.LayoutParams` to draw a semi-transparent image view on top of everything.
- The camera app runs behind this window (assuming user launches camera manually).
- Overlay opacity is adjustable via UI slider.
- Reposition and scaling done using gesture detectors (`ScaleGestureDetector`, `GestureDetectorCompat`).

=== Note

Due to Android OS restrictions: * Drawing overlays over secure content like the camera may not work on all devices or may require `SYSTEM_ALERT_WINDOW` permission. * The user must explicitly grant overlay permissions (draw over other apps).

== Implementation

=== 1. Project Setup

- Create a new Android project using Kotlin.
 - Target Android SDK 30+.
 - Request `SYSTEM_ALERT_WINDOW` permission in manifest: + [source.xml]
-

<uses-permission

android:name="android.permission.SYSTEM_ALERT_WINDOW"/>

=== 2. Image Selection

- Create an `ImageSelectorActivity`.
- Use `Intent.ACTION_PICK` to allow user to choose an image from the gallery.
- Store the URI of the selected image in `SharedPreferences` or pass to the next activity.

=== 3. Overlay Preview Activity

- Create `OverlayPreviewActivity`.
- Configure the activity to draw over other apps: + [source,kotlin]

```
window.setFlags(                                WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL,  
WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL )
```

- Load the selected image as an `ImageView` or `Compose Image` with alpha.
- Provide a transparent background to allow camera to remain visible behind.

=== 4. Overlay Controls

- Implement:
- Opacity slider (`SeekBar` or `Slider`)
- Drag (via `GestureDetector`)
- Scale (via `ScaleGestureDetector`)
- Attach these to the overlay image view.

=== 5. Permission Flow

- On first run, direct user to grant overlay permission using: + [source,kotlin]

```
val intent = Intent(Settings.ACTION_MANAGE_OVERLAY_PERMISSION) intent.data = Uri.parse("package:  
$packageName") startActivity(intent)
```

- Confirm permission before launching `OverlayPreviewActivity`.

=== 6. User Instructions

- Add optional UI overlay with:
- Grid lines (e.g., 3x3 rule of thirds)
- Instructions like "Align and then open your camera manually"

=== 7. Testing

- Test on multiple devices to validate whether overlays appear over vendor camera apps.

- Handle edge cases where camera preview blocks overlays (e.g., secure window flag).

== Milestones

1. Project Initialization [Day 1-2]
2. Set up Android project with required dependencies and permissions.
3. Implement gallery image selection functionality.
4. Overlay Preview UI [Day 3-5]
5. Build `OverlayPreviewActivity` with full-screen layout and transparent background.
6. Implement overlay loading and basic positioning.
7. Overlay Controls [Day 6-8]
8. Add opacity slider.
9. Add gesture support for drag and pinch-zoom.
10. Permission Handling [Day 9]
11. Implement overlay permission request and check.
12. Test behavior on supported devices.
13. Final Polishing [Day 10-11]
14. Add optional alignment grid.
15. Improve UI responsiveness and instructions.
16. Testing & Device Compatibility [Day 12-14]
17. Manual testing on known supported vendor devices (e.g., Pixel, OnePlus).
18. Document any edge cases or unsupported behavior.
19. Release Prep [Day 15]
20. Package, sign APK.
21. Prepare README and usage instructions.

== Gathering Results

To evaluate whether the app meets its goals, the following post-launch metrics and qualitative feedback should be gathered:

=== Functional Validation

- Users are able to select a prior image and visually align it with the real-world scene.

- Overlay positioning, scaling, and opacity behave smoothly and predictably.
- Overlay remains visible when user manually opens the system camera app on supported devices.

=== User Feedback

- Collect early feedback via:
- Internal testing with ~5–10 users from the target audience (e.g., hobby photographers, site surveyors).
- Simple in-app survey link or external form to rate ease of alignment (1–5 scale).
- Questions such as:
 - “Was the overlay visible when you opened your camera?”
 - “How easy was it to align the overlay with the current scene?”
 - “Would you use this for repeated photo captures?”

=== Success Criteria

- 80%+ of users report successful alignment and visibility of overlay during capture.
- No critical bugs on supported devices.
- App rated “easy” or better by majority of testers.

=== Limitations Acknowledged

- The app intentionally avoids handling the camera capture flow, relying on user behavior.
- Known unsupported devices are documented in the app description.